

Αλγόριθμοι και Πολυπλοκότητα

2ο μάθημα

Άσκηση 1.

Το $3^{\log n + n}$ μπορεί να γραφεί ως: $2^{\log 3^{\log n + n}}$
 $= 2^{(\log n + n) \log 3}$. Αρα τώρα είναι αποδεκτό να
 συγκρίνουμε $n!$ και $(\log n + n) \log 3$

- $\log(n!) = \Theta(n \log n)$

- $\log((\log n + n) \log 3) = \log(\log n + n) + \log(\log 3) = \Theta(\log n)$

Αρα $\log(n!) = \omega(\log((\log n + n) \log 3))$,

Οποότε $\forall n \geq n_0$, $\log(n!) > \log((\log n + n) \log 3)$
 $n! > (\log n + n) \log 3$
 $2^{n!} > 2^{(\log n + n) \log 3}$

Άσκηση 2.

$$n^k = O(\log n^{\log n}) = 2^{\log(\log n^{\log n})} = 2^{\log n (\log \log n)}$$

Για να ισχύει η ισότητα πρέπει $\lim_{n \rightarrow \infty} \frac{n^k}{2^{\log n \cdot \log \log n}} = 0 \Rightarrow$

$$\lim_{n \rightarrow \infty} \frac{2^{\log n^k}}{2^{\log n \cdot \log \log n}} = \lim_{n \rightarrow \infty} 2^{k \log n - \log n \cdot \log \log n} = \infty$$

$$\text{After } \lim_{n \rightarrow \infty} (K \log n - \log n \cdot \log(\log n)) = -\infty =$$

$$\Rightarrow \lim_{n \rightarrow \infty} \log n (K - \log(\log n)) = \lim_{n \rightarrow \infty} \frac{K - \log(\log n)}{\frac{1}{\log n}} =$$

$$\stackrel{\text{DLH}}{=} \lim_{n \rightarrow \infty} \frac{0 - \frac{1}{\log n} \cdot \frac{1}{n}}{\frac{1}{(\log n)^2} \cdot \frac{1}{n}} = \lim_{n \rightarrow \infty} -\log n = -\infty$$

Άσκηση 7

1. $i = j = 0$	$i \leq \log n$ Από κάθε επανάληψη
2. while $i \leq \log n$ do	επανάληψη, η τιμή του βρόχου του
3. $i = i + 1$	i γίνεται K . Από $K = \log n$
4. if i περιττός then	Συνολικά η ποσότητα του 1 ^{ου} αλγορίθμου είναι $O(\log n)$
5. $j = j + i$	
6. else	
7. $j = j + i^2$	

Στον 2^ο αλγόριθμο η εξωτερική επανάληψη εκτελείται $\log n$ φορές διότι έχουμε $\frac{n}{2}, \frac{n}{2^2}, \frac{n}{2^3}, \dots, \frac{n}{2^K}$.
 Η for εκτελείται n φορές διότι έχουμε $K=1$ μέχρι $n/2$ βήματα.

Η while περιέχει 1 συνθήκη όπου το j αγγίζει:

$2^0, 2^1, 2^2, \dots, 2^K$, Κ επανάληψεις δηλαδή, μέχρι να φτάσει στο S , όπου το S είναι $1 \cdot 2 \cdot \dots \cdot n = n!$.

Από όπου να βρούμε $2^K = n! \Rightarrow K = \log(n!)$,
 όπου $O(\log(n!)) = O(n \log n)$.

Από έχουμε

$$O(\log(n + n \log n)) = O(n \log n + n \log^2 n) = O(n \log^2 n)$$

Ασκηση 10.

Κατά την ταξινόμηση σε αυξανούσα σειρά, η heapsort πρώτα επιλέγει το μεγαλύτερο στοιχείο και το τοποθετεί στο τέλος της λίστας. Έτσι το στοιχείο που έχει επιλεγεί πρώτο, γίνει τελευταίο και το 2^ο επιλεγμένο γίνει στο προτελευταίο στοιχείο της λίστας. Με αυτόν τον τρόπο όμως, η ταξινόμηση αυτή δεν είναι ευσταθής. π.χ. η λίστα $[21, 20a, 20b, 12, 11, 8, 7]$ περιέχει 2 ίδιες τιμές, το $20a$ και $20b$ (με τα a και b ανακριστούν από είναι πρώτο στην λίστα). Υποθέτουμε την heapsort: $[7, 8, 11, 12, 20b, 20a, 21]$, αλλάζοντας την θέση ίδιων τιμών

Ασκηση 6

$$v.s.o \ O(n^{3/2} \sqrt{\log n}) \cap \omega(\sqrt{n} \log^2(n)) \neq \emptyset$$

Πρέπει να αποδείξουμε ότι υπάρχει συνάρτηση ώστε να ισχύει αυτό που να $f(n)$ είναι το μέγεθος υποσυνόλου. Έστω $f(n) = \sqrt{n} \log n$. πηλεί v.s.o.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^{3/2} \sqrt{\log n}} = 0 \text{ και } \lim_{n \rightarrow \infty} \frac{f(n)}{\sqrt{n} \log^2 n} = 0$$

$$\textcircled{1} \lim_{n \rightarrow \infty} \frac{f(n)}{n^{3/2} \sqrt{\log n}} = \lim_{n \rightarrow \infty} \frac{\sqrt{n} \log n}{n^{3/2} \sqrt{\log n}} = \lim_{n \rightarrow \infty} \frac{\sqrt{\log n}}{n} \stackrel{\infty}{=} 0 \quad \text{DL'H}$$

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2} \sqrt{\log n} \cdot \frac{1}{n}}{1} = \lim_{n \rightarrow \infty} \frac{1}{2 \sqrt{\log n} \cdot n} \stackrel{\frac{1}{\infty}}{=} 0$$

$$\textcircled{2} \lim_{n \rightarrow \infty} \frac{\sqrt{n} \log n}{\sqrt{n} \log^2 n} = \lim_{n \rightarrow \infty} \frac{1}{\log n} = 0$$

Αρα υποθέτουμε σε $\sqrt{n} \log n$ ανήκει και στο $O(n \sqrt{\log n})$ και στο $\omega(\sqrt{n} \log^2 n)$, ορα το συμπό-
 των 2 συνάρτησεων δεν είναι κενό

Άσκηση 8

Αλγόριθμος CheckPerm

1. Για κάθε i αντιστάθιση
2. Ελέγχεται αν ο πίνακας A είναι ίσος με τον B
3. Επιστρέφει True ή False

Το βήμα 2 απαιτεί $O(n)$ χρόνο

Το βήμα 1 έχει έναν βρόχο που θα εκτελέσει το ποσό $n + (n-1) + (n-2) + \dots = \frac{n \cdot (n-1)}{2}$ δηλαδή

απαιτεί $O(n^2)$ χρόνο. Δεδομένου ότι ο έλεγχος αν ο A και ο B είναι ίσοι απαιτεί $O(n)$ χρόνο, η πολυπλοκότητα του CheckPerm είναι $O(n^3) = O(T(n))$

Ο καλύτερος χρόνος εκτέλεσης θα συμβεί όταν απαιτείται μόνο 1 αντιστάθιση, ώστε ο πίνακας A να ισούται με τον B . Άρα $O(T(n)) = O(n)$

Άσκηση 9

Σε ένα σωρό ελαχιστών, ο γονέας είναι πάντα μικρότερος από το παιδί. Ως εκ τούτου, τα φύλλα περιέχουν την μέγιστη τιμή. Γνωρίζοντας ότι τα φύλλα είναι n και οι ελαχιστοί κόμβοι (έστω $n/2$), τότε χρειαζόμαστε να αναζητήσουμε $n/2$ κόμβους.

Άρα $O(n/2) = O(n)$ ο χρόνος εύρεσης μέγιστου σε σωρό ελαχιστών.